Программирование для Интернет

Лекция 6. AJAX

Содержание лекции:

- Принцип работы AJAX
- AJAX и JavaScript
- JSON
- AJAX и Jquery
- AJAX и PHP

Преподаватель курса: Нефедов Денис Геннадьевич, к.т.н., доцент

Принцип работы AJAX

AJAX – Asynchronous JavaScript And XML (Асинхронный JavaScript и XML).

- Это способ совместного использования HTML, CSS, JavaScript и DOM.
- Использование AJAX заметно увеличивает скорость реакции интерфейса и значительно уменьшает нагрузку на сервер. Это становится возможным благодаря асинхронному обмену информацией и способностью перезагружать только "обновленную" часть страницы без необходимости перезагрузки страницы целиком.

AJAX - это набор технологий, которые поддерживаются веб-браузерами. AJAX использует:

- HTML в качестве "каркаса";
- CSS для оформления;
- DOM для извлечения или изменения информации на странице;
- Объект XMLHttpRequest для асинхронного обмена данными с сервером;
- JavaScript для связи перечисленных выше технологий между собой.

AJAX и JavaScript

Этапы выполнения AJAX запроса:

I. Создание экземпляра объекта XMLHttpRequest

II. Отправка запроса на сервер с помощью методов open и send

III. Принятие сервером запроса, обработка и отправление запрашивающему

IV. Принятие и обработка ответа сервера с помощью responseText, responseXML и onreadystatechange

I. AJAX-запросы совершаются с помощью объекта XMLHttpRequest.

Создание экземпляра объекта XMLHttpRequest

```
переменная=new ActiveXObject("Microsoft.XMLHTTP");
```

Свойства объекта XMLHttpRequest

onreadystatechange	Функция, код которой будет исполнен при изменении состояния готовности сервера.
readyState	Позволяет узнать состояние готовности сервера.
responseText	Хранит ответ сервера как строку символов.
responseXML	Хранит ответ сервера как XML файл.
status	Хранит код ответа сервера.

Методы объекта XMLHttpRequest:

open(тип_запроса,url,способ)	Создает запрос. тип_запроса устанавливает тип запроса (GET или POST). url устанавливает путь к запрашиваемому файлу. способ устанавливает способ выполнения запроса. При значении true запрос будет выполнен асинхронно, при false синхронно.
send('данные')	Позволяет передать данные на сервер.
setRequestHeader(заголовок, значение)	Позволяет добавить HTTP заголовок к запросу. заголовок содержит имя заголовка значение содержит значение заголовка

II. AJAX запросы отправляются с помощью методов open() и send() объекта XMLHttpRequest

```
// Создаем объект XMLHttpRequest
var xhttp=new XMLHttpRequest();
// Запросим содержимое файла testfile.txt
xhttp.open('GET',testfile.txt,true);
xhttp.send();
```

Методы

open(тип_запроса,url, способ)	Создает AJAX запрос. тип_запроса устанавливает тип запроса (GET или POST). url устанавливает путь к запрашиваемому файлу. способ устанавливает способ выполнения запроса. При значение true запрос будет выполнен асинхронно, при false синхронно.
send('данные')	Позволяет передать данные на сервер.

Запрос GET/POST

Пример запроса содержимого файла testfile.txt и вывод его на страницу

```
xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
   if (xhttp.readyState==4 && xhttp.status==200)
     document.getElementById("result").innerHTML=xhttp.responseText;
   }
xhttp.open("GET","testfile.txt",true);
//Или для метода POST
//xhttp.open("POST","testfile.txt",true);
xhttp.send();
```

Параметры POST запроса не отображаются в адресной строке браузера и он более подходит при передаче

- секретных данных (такие как логин и пароль пользователя);
- большого количества данных. GET имеет предел на количество передаваемых данных (2000 символов), а POST нет;
- получении некэшированного результата

С помощью свойства readyState можно узнать состояние готовности сервера.

Сервер может находится в 5 состояниях готовности:

- Запрос не инициализирован;
- Установлено подключение к серверу;
- Запрос получен;
- Обработка запроса;
- Обработка запроса закончена и ответ готов.

Во время обработки запроса свойство readyState поочередно принимает значения от 0 до 4.

С помощью свойства status можно узнать код ответа на Ваш запрос.

Код 200 означает - запрос обработан успешно, код 404 означает - страница не существует.

Пример определения статуса страницы.

```
function requestGood(){
   var xhttp=new XMLHttpRequest();
   xhttp.onreadystatechange=function(){
      document.getElementById('result').innerHTML=xhttp.status;
   }
   xhttp.open('GET','test.xml',true);
   xhttp.send();
}
```

Событие onreadystatechange выполняет код при изменении содержимого свойства readyState (т.е. во время успешного запроса код функции выполняется 4 раза).

Для того, чтобы корректно обработать ответ сервера необходимо добавить проверку условия if (XMLHttpRequest.readyState==4 && XMLHttpRequest.status==200) и только после этого получать и обрабатывать соответствующие данные.

III. Чтобы принять ответ сервера используются свойства объекта XMLHttpRequest

responseText	Хранит ответ сервера как строку символов.
responseXML	Хранит ответ сервера как XML файл.

Если запрошенные у сервера данные не являются XML данными, то для считывания ответа сервера используется свойство responseText.

```
xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
  if (xhttp.readyState==4 && xhttp.status==200)
  document.getElementById("ajax").innerHTML=xhttp.responseText;
  }
xhttp.open("GET","testfile.txt",true);
xhttp.send();
```

IV. Для получения от сервера XML данных используется для считывания ответа сервера свойство responseXML.

```
xhttp=new XMLHttpRequest();
xhttp.onreadystatechange=function(){
if (xhttp.readyState==4 && xhttp.status==200){
      var r=xhttp.responseXML;
      document.getElementById('ajax').innerHTML="Содержимое первого
      тэга name в XML файле: "+r.getElementsByTagName('name')[0].
      childNodes[0].nodeValue;
      var dep=r.getElementsByTagName('dept');
      var cont="Содержимое всех тэгов dept в XML файле: <br />";
      for (var i=0;i<dep.length;i++){
      cont+=(i+1)+". "+dep[i].childNodes[0].nodeValue+" руб.<br />";
      }
document.getElementById('ajax1').innerHTML=cont;
}
}
xhttp.open("GET","test.xml",true);
xhttp.send();
```

JSON

JSON расшифровывается JavaScript Object Notation.

Это простой и универсальный формат передачи данных и может использоваться с AJAX вместо XML.

Преимущества JSON:

- Простота использования;
- Упрощенный синтаксис обеспечивающий легкость прочтения кода.

Преимущества XML:

- XML поддерживается почти всеми языками программирования;
- XML расширяем.

JSON включает в себя следующие структуры:

- Массив - это список упорядоченных значений. Массив в JSON обозначается квадратными скобками [] с неограниченным количеством значений.

```
[значение1, значение2, значениеN]
```

- **Объект** может содержать неограниченное количество пар имя/значения. Каждое из значений само может являться объектом или массивом. Такие объекты или массивы называются вложенными.

```
{имя1:значение1,
имя2:
{имя2_1:значение2_1,имя2_2:значение2_2}
}
```

- Объекты и массивы в JSON являются конструкциями, а **литералы** непосредственно данными, которые группируются этими конструкциями.

```
{имя1:"строка", имя2:13, имя3:true, имя4:false, имя5:null}
```

  - Строка;
  - Число;
  - Логическое значение;
  - Значение null.

Сравнение файлов XML и JSON, содержащих описание книг в книжном магазине.

bookbase.json	bookbase.xml

`{`
`bookbase:[`
`{`
`title:"Гарри Поттер и философский камень",`
`author:"Джоан Ролинг",`
`cost:500,`
`quantity:15`
`},`
`{`
`title:"Дракула",`
`author:"Брэм Стокер",`
`cost:430,`
`quantity:5`
`},`
`{`
`title:"Преступление и наказание",`
`author:"Федор Достоевский",`
`cost:400,`
`quantity:10`
`},`
`.....`
`]`
`}`
	`<?xml version="1.0" encoding="UTF-8" ?>`
	`<bookstore>`
	`<book id='1'>`
	`<title>Гарри Поттер и философский камень`
	`</title>`
	`<author>Джоан Ролинг</author>`
	`<cost>500</cost>`
	`<quantity>15</quantity>`
	`</book>`
	`<book id='2'>`
	`<title>Дракула</title>`
	`<author>Брэм Стокер</author>`
	`<cost>430</cost>`
	`<quantity>5</quantity>`
	`</book>`
	`<book id='3'>`
	`<title>Преступление и наказание</title>`
	`<author>Федор Достоевский</author>`
	`<cost>400</cost>`
	`<quantity>10 </quantity>`
	`</book>`
	`.....`
	`</bookstore>`

Пример. Запрос содержимого нескольких тэгов файла bookbase.json из JavaScript.

Запросить содержимое

Название книги: **Дракула**

Автор: **Брэм Стокер**

Цена: **430 руб.**

Количество на складе: **5 шт.**

```
xhttp=new XMLHttpRequest();
xhttp.open('GET','bookbase.json',true);
xhttp.send();
xhttp.onreadystatechange=function()
{
   if (xhttp.readyState==4)
      {
      //Принятое содержимое json файла должно быть вначале обработано
функцией eval
      var json=eval( '('+xhttp.responseText+')' );
      //Далее мы можем спокойно использовать данные
      document.getElementById('wrap').style.dislay='block';
      document.getElementById('res1').innerHTML=json.bookbase[1].title;
      document.getElementById('res2').innerHTML=json.bookbase[1].author;
      document.getElementById('res3').innerHTML=json.bookbase[1].cost;
      document.getElementById('res4').innerHTML=json.bookbase[1].quantity;
      }
}
```

JQuery

Это библиотека JavaScript, позволяющая упростить и ускорить написание кода.

Для того, чтобы начать использовать jQuery необходимо:

- Скачать ее с официального сайта http://jquery.com.
- Добавить ее на страницу. Для этого следующий код должен быть добавлен на страницу в секцию head:

```
<script type="text/javascript" src="путь_к_скачанному_файлу/jquery.js">
</script>
```

Для удаленного доступа (без скачивания)

```
<script type="text/javascript"
src="http://ajax.googleapis.com/ajax/libs/jquery/1.5/jquery.min.js"></s
cript>
```

Стандартный синтаксис jQuery команд:

```
$(селектор).метод();
```

1. Знак $ сообщает, что символы идущие после него являются jQuery кодом;
2. Селектор позволяет выбрать элемент на странице;
3. Метод задает действие, которое необходимо совершить над выбранным элементом. Методы в jQuery разделяются на следующие группы:
   - Методы для манипулирования DOM;
   - Методы для оформления элементов;
   - Методы для создания AJAX запросов;
   - Методы для создания эффектов;
   - Методы для привязки обработчиков событий.

jQuery можно комбинировать с обычным JavaScript.
Если строка начинается с $ - это jQuery, если $ вначале строки отсутствует - это строка JavaScript кода. С помощью команды noConflict() можно заменить знак $ на любой другой.

Пример.

```
//Изменим стандартный символ $ на nsign
var nsign=jQuery.noConflict();
//Изменим цвет абзаца используя nsign вместо $
nsign(document).ready(function(){

   nsign("p").css("color","green");

});
```

С помощью селекторов можно выбирать элементы на странице для применения к ним определенных действий.

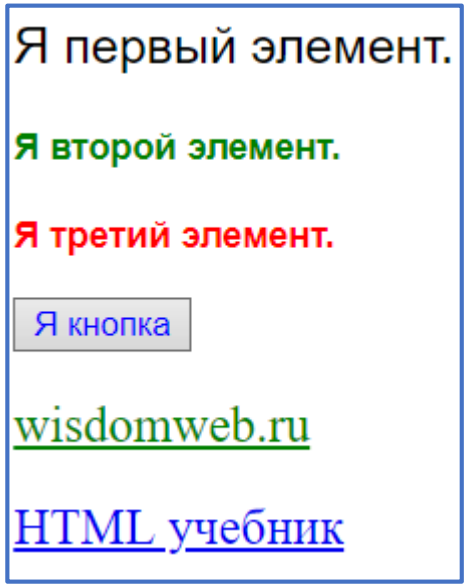
`$("p")`	Будут выбраны все элементы p, которые находятся на странице.
`$(".par")`	Будут выбраны все элементы на странице с class="par".
`$("#par")`	Будет выбран первый элемент на странице с id="par".
`$(this)`	Позволяет выбрать текущий HTML элемент. Щелкните на $(this) слева, чтобы посмотреть пример использования данного селектора в онлайн редакторе.

$("p.par")	Будут выбраны все элементы p на странице с class="par".
$("p#par")	Будут выбраны все элементы p на странице с id="par".
$(".par,.header,#heat")	Будут выбраны все элементы на странице со значениями атрибутов class="par", class="header" и id='heat'.
$("[src]")	Будут выбраны все элементы на странице, имеющие атрибут src.
$("[src='значение']")	Будут выбраны все элементы со значениям атрибута src="значение".
$("[src$='.gif']")	Будут выбраны все элементы со значениями атрибута src заканчивающимися на .gif.
$("div#wrap .par1")	Будут выбраны все элементы с class=par1, которые находятся внутри элементов div с id=wrap.
$(":input")	Будут выбраны все input элементы на странице.
$(":тип")	Будут выбраны все input элементы с <input type='тип' />. Например :button выберет все <input type='button' /> элементы, :text выберет все <input type='text' /> элементы и т.д.

Пример.

```
$(document).ready(function(){
//Установим размер шрифта всех абзацев равным 20 пикселям
   $("p").css("fontSize","20px");
//Изменим на зеленый цвет шрифта элемента с id=el2
   $("#el2").css("color","green");
//Изменим на красный цвет шрифта элемента с class=el3
   $(".el3").css("color","red");
//Сделаем жирным шрифт элементов с id=el2 и class=el3
   $("#el2,.el3").css("fontWeight","bold");
//Изменим на синий цвет текста кнопки
   $(":input").css("color","blue");
//Установим размер шрифта всех элементов имеющих атрибут
href равным 20 пикселям
   $("[href]").css("fontSize","20px");
//Изменим на зеленый цвет ссылки на www.wisdomweb.ru

$("[href='http://www.wisdomweb.ru/']").css("color","green
");
});
```



В jQuery можно избавиться от преждевременного выполнения кода следующими способами:

```
<script type='text/javascript'>
//Первый способ:
$(document).ready(function(){
   Код который будет выполнен после полной загрузки документа
});

//Второй способ:
$().ready(function(){
   Код который будет выполнен после полной загрузки документа
});

//Третий способ:
$(function(){
   Код который будет выполнен после полной загрузки документа
});
</script>
```

Для того, чтобы сократить размер кода можно соединять команды jQuery в цепочки.

Команды в цепочке будут выполняться поочередно слева направо.

```
<script type='text/javascript'>
//Код без сокращения
$("p").css("color","green");
$("p").css("font-size","30px");
//Сокращенный код
$("p").css("color","green").css("font-size","30px");
</script>
```

Обработчики событий - это функции, код которых исполняется только после совершения определенных действий.

```
$(селектор).обработчик_события(function(){код_обработчика_события});
```

Код обработчика **mouseover** будет выполнен, когда курсор мыши будет наведен на элемент.

Код **mouseout** - когда курсор мыши будет выведен из границ элемента.

```
$(document).ready(function(){
  $("p").mouseover(function(){$("p").css("color","green")});
  $("p").mouseout(function(){$("p").css("color","black")});
});
```

Код **click** будет выполнен после одинарного щелчка мыши на элементе.

Код **dblclick** будет выполнен после двойного щелчка мыши на элементе.

```
$(document).ready(function(){
  $("#but1").click(function(){alert("Вы нажали один раз на первую кнопку!");});
  $("#but2").dblclick(function(){alert("Вы нажали два раза на вторую
кнопку!");});
});
```

Код focus() будет выполнен, когда элемент станет активным.

Код blur() будет выполнен, когда элемент перестанет быть активным.

Код change() будет выполнен, при изменении содержимого элемента.

```
$(document).ready(function(){
   $("#el1").focus(function(){$(this).attr("value","")});
   $("#el1").blur(function(){$(this).attr("value","Введите ФИО")});
   $("#el2").change(function(){ alert("Содержимое данного элемента было
изменено.") });
});
```

С помощью jQuery также вызываются обработчики событий привязанные к элементу. Например: $('#test').blur() вызовет обработчик blur у элемента с id='test'.

```
$(document).ready(function(){
// Зададим обработчик события, который будет выводить сообщение при нажатии на
кнопку с id=but1
   $("#but1").click(function(){alert("Вы нажали на кнопку с id=but1")});
// Вызовем обработчик click у элемента с id='but1'
   $("#but1").click();
});
```

Объект event содержит информацию о произошедшем событии.

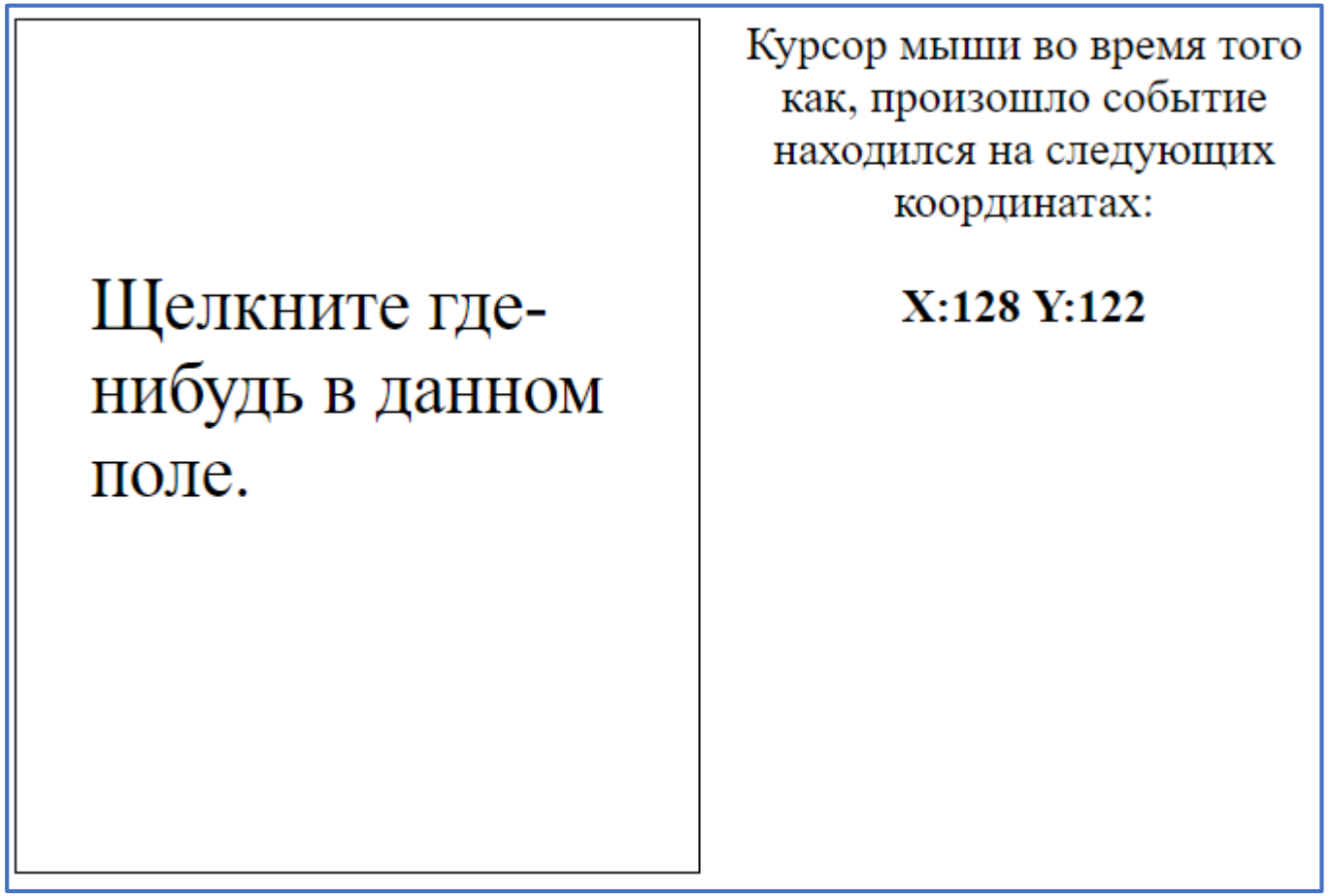
$(селектор).событие(function(event){код_обработчика_события});

В примере с помощью объекта event выводятся координаты, на которых находился курсор мыши во время того, как произошло событие.

```
$(document).ready(function(){

  $("#square").click(function(event){
    $("#coord").css("display","block");
    $("#x").html(event.pageX);
    $("#y").html(event.pageY);
  });

});
```



Метод jQuery one() позволяет создавать обработчики, которые могут быть вызваны лишь один раз.

```
$(document).ready(function(){
  $("#but1").one("click",function(){alert("Вы нажали на кнопку с id=but1")});
});
```

Метод jQuery toggle() позволяет переключаться между различными обработчиками событий по щелчку мыши.

```
$(document).ready(function(){
  $("#but1").click(function(){
    $("#par1").css("color","black");
    $("#par1").css("font-size","1em");
  });
  $("#par1").toggle(function(){
    $("#par1").css("color","green");
    $("#par1").css("font-size","10px");},function(){
    $("#par1").css("color","red");
    $("#par1").css("font-size","20px");}, function(){
    $("#par1").css("color","black");
    $("#par1").css("font-size","1em");
  });
});
```

Щелкните на содержимом данного абзаца для того, чтобы переключить оформление. При повторных нажатиях абзац будет оформляться по разному.

Сбросить оформление на стандартное

С помощью jQuery методов fadeOut(), fadeIn() и fadeTo() можно постепенно скрывать и отображать элементы анимировано.

```
//Позволяет постепенно скрыть выбранный элемент
$("селектор").fadeOut(скорость,функция);
//Позволяет постепенно отобразить выбранный элемент
$("селектор").fadeIn(скорость,функция);
//Позволяет постепенно скрыть/отобразить элемент до указанного значения прозрачности
$("селектор").fadeTo(скорость,прозрачность,функция);
```

С помощью jQuery методов slideUp, slideDown и slideToggle можно плавно изменять высоту выбранных элементов.

```
//Позволяет изменяет высоту элемента до 0
$("селектор").slideUp(скорость,функция);
//Позволяет плавно возвратить элементу его изначальную высоту
$("селектор").slideDown(скорость,функция);
//При первом вызове будет действовать как slideUp, а при втором как slideDown
$("селектор").slideToggle(скорость,функция);
```

С помощью метода slideToggle можно создавать на страницах удобные выпадающие меню.

```
$(document).ready(function(){
$("#menu").click(function(){$("#list").slideToggle(
2000)});
  $("#menu").toggle(function(){
$("#img").attr("src","menudown.gif")},function(){
    $("#img").attr("src","menuup.gif")
  });
$("#menu").mouseover(function(){$("#menu").css
("background-color","#01939a")});

$("#menu").mouseout(function(){$("#menu").css(
"background-color","#006064")});
});
```

Щелкните на заголовок "выбрать язык" для того, чтобы свернуть или развернуть меню.

Выбрать язык ▲

Английский язык

Русский язык

Французский язык

Испанский язык

Немецкий язык

С помощью метода animate() можно создавать на страницах полноценную анимацию.

$("селектор").animate({стили},скорость,функция_смягчения,функция);

- стили задает CSS стили для анимации (к элементу одновременно может быть применено несколько стилей).
- скорость задает скорость анимации. Вы можете указать скорость используя предопределенные свойства: "slow", "fast", "normal" (медленно, быстро, нормально) или указать скорость в миллисекундах (1000 миллисекунд = 1 секунда).
- функция_смягчения задает функцию, которая будет отвечать за плавность выполнения анимации.
- функция указывает имя функции, код которой будет выполнен после завершения анимации.

Пример.

```
$(document).ready(function(){

  $("#but1").click(function(){
    $("p").animate({fontSize:30},2000);
    $("p").animate({top:220},2000);
    $("p").animate({fontSize:"1em"},2000);
    $("p").animate({left:320},2000);
    $("p").animate({top:0,left:0},2000);
  });

});
```

Jquery DOM.

С помощью метода html() можно изменить или узнать внутреннее содержимое выбранного элемента.

Синтаксис:

```
//Узнаем содержимое выбранного элемента
$("селектор").html();
//Изменим содержимое выбранного элемента
$("селектор").html("новое содержимое");
```

С помощью метода append() - вставить произвольный текст после внутреннего содержимого выбранного элемента.

С помощью метода prepend() - вставить произвольный текст перед внутренним содержимым выбранного элемента.

С помощью метода attr() - узнать или изменить содержимое указанного атрибута у выбранного элемента.

С помощью метода removeAttr() - удалить указанный атрибут у выбранного элемента.

Jquery CSS.

С помощью метода css() можно узнавать текущие или устанавливать новые значения свойств оформления элементов.

```
//Узнаем значение указанного CSS свойства выбранного элемента
$("селектор").css("свойство");
//Установим новое значение указанному CSS свойству выбранного элемента
$("селектор").css("свойство","значение");
//Установим произвольные значения нескольким CSS свойствам выбранного элемента
$("селектор").css({свойство1:значение1, свойствоN:значениеN});
```

С помощью метода addClass - добавить указанный класс выбранному элементу (данный метод, добавляя новый класс, не удаляет и не заменяет существующие).

С помощью метода removeClass - удалить указанный класс у выбранного элемента.

Метод toggleClass позволяет переключаться между удалением и добавлением класса выбранного элемента.

Jquery Плагины.

Разработчик может написать плагин, который позволяет реализовать определенную функцию и затем использовать его в необходимых приложениях написав только одну строчку кода.

Для того, чтобы создать плагин необходимо добавить к объекту jQuery.fn свойство, имя которого будет является именем плагина:

```
//Определяем код плагина
(function($){
  $.fn.имяПлагина = function() {
    // Код плагина
  };
})(jQuery);
//Вызываем плагин
$(селектор).имяПлагина();
```

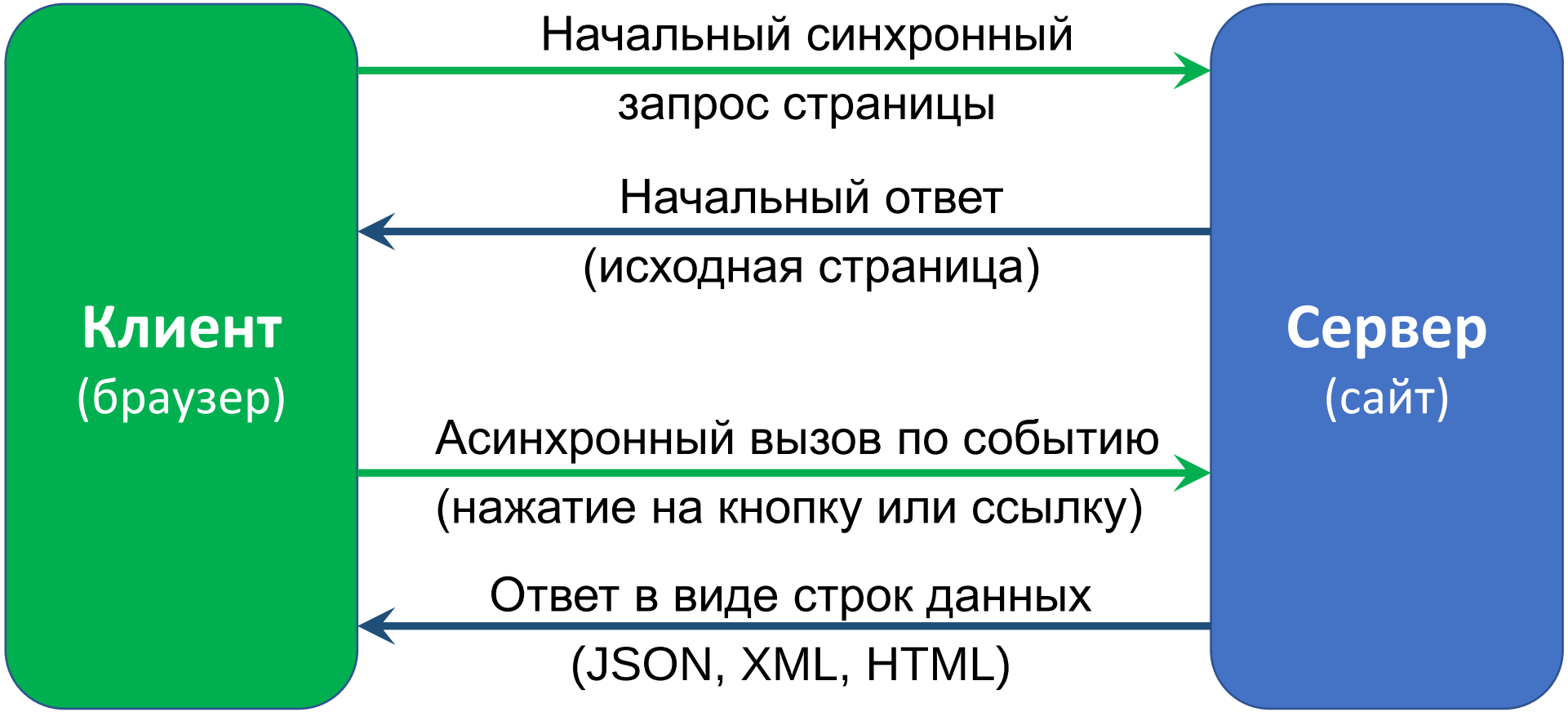
Заключительным шагом в создании плагинов должно стать вынесение кода плагина в отдельный файл. И написания инструкций, которые должны разъяснить потенциальным пользователям как и зачем можно использовать Ваш плагин.

```
<script src="http://www.wisdomweb.ru/editor/plugin1.js"></script>

$(document).ready(function(){
  $("#but1").click(function(){
    $("#demo p").adjust();
  });
  $("#but2").click(function(){
    $("#demo p").adjust({size:"2em"});
  });
  $("#but3").click(function(){
    $("#demo p").adjust({size:"1.7em",color:"green"});
  });
});
```

AJAX и PHP

Общая схема работы AJAX.



При формировании запроса на стороне браузера используется функция $.ajax, которая также имеет сокращенные форматы $.post и $.get – в зависимости от типа запроса.
Пример ajax запроса с использованием jquery и его составные части

```
<!-- index.php -->
<?php
$.ajax({
    url: '/file.php',        /* Куда пойдет запрос */
    method: 'get',           /* Метод передачи (post или get) */
    dataType: 'html',        /* Тип данных в ответе (xml, json, script, html). */
    data: {text: 'Текст'},   /* Параметры передаваемые в запросе. */
    success: function(data){/* функция выполнится после успешного запроса.  */
        alert(data);         /* В переменной data ответ от index.php. */
    }
});
?>
```

Данный запрос части делает то же самое, что и переход в браузере по ссылке

http://site.com/file.php?text=Текст

- url – ссылка на обработчик, если он находится в том же файле, что и форма, то пишется адрес этой же страницы
- method – метод (post, get)
- datatype – тип данных в ожидающемся ответе от сервера (json, html, script или др.)
- data – данные, передаваемые на сервер в виде массива
- success – действие, выполняемое после успешного выполнения задачи.

Сокращенная запись

```
$.get('/file.php', {text: 'Текст'}, function(data){
    alert(data);
});
```

Пример ответа от сервера

```
// file.php
echo 'Данные приняты - ' . $_GET['text'];
```

Чтобы get-запросы не кэшировались, ставится соответствующая опция

```
$.ajax({
    url: '/index.php',
    method: 'get',
    cache: false
});
```

В post-запросе изменяется get на post в параметрах или названии функции:

```
$.post('/index.php', {text: 'Текст'},
function(data){
    alert(data);
});
```

Рассмотрим пример обработки входа пользователя.
Создадим форму для ввода логина и пароля в файле Index.php

```
<!doctype html>
<html>
<head>
<script src="https://code.jquery.com/jquery-3.3.1.js" integrity="sha256-
2Kok7MbOyxpgUVvAk/HJ2jigOSYS2auK4Pfzbm7uH60=" crossorigin="anonymous"></script>
</head>
<body>
<form id="loginform" method="post">
    <div>
        Username:
        <input type="text" name="username" id="username" />
        Password:
        <input type="password" name="password" id="password" />
        <input type="submit" name="loginBtn" id="loginBtn" value="Login" />
    </div>
</form>
<script type="text/javascript">
```

и запишем функцию отправки, вызываемую при отправке данных с формы

```
$(document).ready(function() {
    $('#loginform').submit(function(e) {
        e.preventDefault();
        $.ajax({
            type: "POST",
            url: 'login.php',
            data: $(this).serialize(),
            success: function(response)
            {
                var jsonData = JSON.parse(response);
                if (jsonData.success == "1")
                {
                    location.href = 'my_profile.php';
                }
                else
                {                    alert('Invalid Credentials!');                }
            }
        });
    });
});
</script>
</body>
</html>
```

Файл сервера login.php содержит логику аутентификации пользователей и возвращает ответ JSON, основанный на успешном или неудачном входе в систему

```php
<?php
if (isset($_POST['username']) && $_POST['username'] &&
isset($_POST['password']) && $_POST['password']) {
    // do user authentication as per your requirements
    // ...
    // ...
    // based on successful authentication
    echo json_encode(array('success' => 1));
} else {
    echo json_encode(array('success' => 0));
}
```

При работе с массивами данных удобно использовать формат json

```
<?php
$.ajax({
    url: '/json.php',
    method: 'get',
    dataType: 'json',
    success: function(data){
        alert(data.text);    /* выведет "Текст" */
        alert(data.error);   /* выведет "Ошибка" */
    }
});
?>
```

Короткая запись

```
$.getJSON('/json.php', function(data) {
    alert(data.text);
    alert(data.error);
});
```

Код файла json.php

```
header('Content-Type: application/json');

$result = array(
    'text'  => 'Текст',
    'error' => 'Ошибка'
);

echo json_encode($result);
```

иногда может потребоваться очистка буфера вывода, в этом случае добавьте

```
// Очистка буфера
ob_end_clean();
header('Content-Type:
application/json');
echo json_encode($result,
JSON_UNESCAPED_UNICODE);
exit();
```

Через ajax также можно подгружать код из javascript, который выполняется сразу после успешного запроса.

```
$.ajax({
    method: 'get',
    url: '/script.js',
    dataType: "script"
});
```

или коротко

```
$.getScript('/script.js');
```

По умолчанию в JQuery AJAX-запросы выполняются асинхронно. Т.е. запрос не задерживает выполнение программы пока ждет результатов, а работает параллельно.

Пример

```
var text = '';
$.ajax({
    url: '/file.php',
    method: 'get',
    dataType: 'html',
    success: function(data){
        text = data;
    }
});
alert(text);  /* Переменная будет пустая. */
```

Чтобы включить синхронный режим нужно добавить параметр async: false. Соответственно синхронный запрос будет вешать прогрузку страницы если код выполняется

```
$.ajax({
    url: '/file.php',
    method: 'get',
    dataType: 'html',
    async: false,
    success: function(data){
        text = data;
    }
});

alert(text); /* В переменной будет результат из file.php. */
```

Через AJAX можно отправить заголовки HEAD, они указываются в параметре headers.

```
$.ajax({
    url: '/index.php',
    method: 'get',
    dataType: 'html',
    headers: {'Token_value': 123},
    success: function(data){
        console.dir(data);
    }
});
```

В PHP они будут доступны в массиве $_SERVER, ключ массива переводится в верхний регистр с приставкой HTTP_, например:

```
<?php
echo $_SERVER['HTTP_TOKEN_VALUE']; // 123
?>
```

Через параметр error задается callback-функция, которая будет вызвана в случае, если запрашиваемый ресурс отдал 404, 500 или другой код.

```
$.ajax({
    url: '/index.php',
    method: 'get',
    dataType: 'json',
    success: function(data){
        console.dir(data);
    },
    error: function (jqXHR, exception) {
        if (jqXHR.status === 0) {
            alert('Not connect. Verify Network.');
        } else if (jqXHR.status == 404) {
            alert('Requested page not found (404).');
        } else if (jqXHR.status == 500) {
            alert('Internal Server Error (500).');
        } else if (exception === 'parsererror') {
            alert('Requested JSON parse failed.');
        }
    }
});
```

Через $.ajaxSetup можно задать обработчик ошибок для всех AJAX-запросов на сайте.

```
$.ajaxSetup({
    error: function (jqXHR, exception) {
        ...
    }
});
```